

# **Twiddle Puzzle Solver**

Using Dynamic Programming and Backtracking

Sivasubramani K J (CB.SC.U4CSE24752)

Samanyu Nair (CB.SC.U4CSE24745)

Shreyas N (CB.SC.U4CSE24750)

March 13, 2026

Course: Design and Analysis of Algorithms (DAA)

Phase 3 Project Report

Instructor: Dr. Vidhya Balasubramanian

Department of Computer Science and Engineering

Amrita Vishwa Vidyapeetham

# Contents

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                              | <b>4</b>  |
| <b>2</b> | <b>Problem Definition</b>                                                        | <b>4</b>  |
| 2.1      | Puzzle Description . . . . .                                                     | 4         |
| 2.2      | State Representation . . . . .                                                   | 5         |
| 2.3      | Initial State . . . . .                                                          | 5         |
| 2.4      | Goal State . . . . .                                                             | 5         |
| 2.5      | Allowed Moves . . . . .                                                          | 5         |
| <b>3</b> | <b>Algorithmic Approaches</b>                                                    | <b>6</b>  |
| 3.1      | Case 7 — Backtracking AI ( <code>BacktrackingPlayer</code> ) . . . . .           | 6         |
| 3.1.1    | Concept . . . . .                                                                | 6         |
| 3.1.2    | Iterative Deepening (IDDFS) . . . . .                                            | 6         |
| 3.1.3    | Sub-Problem Definition . . . . .                                                 | 6         |
| 3.1.4    | Mapping to a Classical Problem . . . . .                                         | 7         |
| 3.1.5    | Pruning Strategies . . . . .                                                     | 7         |
| 3.1.6    | Backtracking Pseudocode . . . . .                                                | 8         |
| 3.1.7    | Greedy Fallback . . . . .                                                        | 9         |
| 3.1.8    | Time and Space Complexity . . . . .                                              | 9         |
| 3.2      | Case 8 — Top-Down DP ( <code>TopDownDPPlayer</code> ) . . . . .                  | 9         |
| 3.2.1    | Concept . . . . .                                                                | 9         |
| 3.2.2    | Sub-Problem Definition . . . . .                                                 | 10        |
| 3.2.3    | Mapping to a Classical Problem . . . . .                                         | 10        |
| 3.2.4    | The Memo Key . . . . .                                                           | 10        |
| 3.2.5    | DP Recurrence . . . . .                                                          | 11        |
| 3.2.6    | Top-Down DP Pseudocode . . . . .                                                 | 11        |
| 3.2.7    | Difference from Case 7 . . . . .                                                 | 12        |
| 3.2.8    | Time and Space Complexity . . . . .                                              | 12        |
| 3.3      | Case 6 — MDF DP ( <code>ComputerPlayer3</code> + <code>DPFlow</code> ) . . . . . | 12        |
| 3.3.1    | Concept . . . . .                                                                | 12        |
| 3.3.2    | Sub-Problem Definition . . . . .                                                 | 13        |
| 3.3.3    | Mapping to a Classical Problem . . . . .                                         | 13        |
| 3.3.4    | Bottom-Up Table Construction . . . . .                                           | 13        |
| 3.3.5    | DP Recurrence Equation . . . . .                                                 | 14        |
| 3.3.6    | Bottom-Up DP Pseudocode . . . . .                                                | 14        |
| 3.3.7    | Query Phase via <code>DPFlow</code> . . . . .                                    | 15        |
| 3.3.8    | Reshuffle Mechanism . . . . .                                                    | 15        |
| 3.3.9    | Time and Space Complexity . . . . .                                              | 15        |
| <b>4</b> | <b>Comparison of DP vs Backtracking</b>                                          | <b>17</b> |
| <b>5</b> | <b>Challenges Faced</b>                                                          | <b>18</b> |
| 5.1      | Large State Spaces . . . . .                                                     | 18        |
| 5.2      | Avoiding Repeated States . . . . .                                               | 18        |
| 5.3      | Correct Memo Key Design (Case 8) . . . . .                                       | 18        |
| 5.4      | Inverse Rotation for Bottom-Up DP (Case 6) . . . . .                             | 18        |
| 5.5      | Thread Safety and Cancellation . . . . .                                         | 18        |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 5.6      | Reshuffle Logic for Partial DP Table . . . . . | 18        |
| <b>6</b> | <b>Conclusion</b>                              | <b>19</b> |

# Abstract

This report presents the design and implementation of a Twiddle Puzzle solver using two core algorithmic techniques: Dynamic Programming (DP) and Backtracking. The Twiddle Puzzle is an  $N \times N$  sliding tile puzzle where the only allowed operation is a  $90^\circ$  counter-clockwise rotation of any  $2 \times 2$  sub-grid, called a *twiddle move*. The objective is to arrange all tiles into ascending order.

Three distinct solvers are implemented and studied: a **Bottom-Up MDF DP** solver (Case 6) that pre-computes an optimal move table via reverse BFS from the goal state; a **Backtracking AI** (Case 7) that uses iterative-deepening depth-first search with heuristic pruning and move ordering; and a **Top-Down DP** solver (Case 8) that combines backtracking with memoisation to avoid redundant sub-problem computation.

Experimental results show that the MDF DP solver achieves  $O(1)$  per-move query time after preprocessing, while the Top-Down DP outperforms pure backtracking on boards with many state transpositions. The project demonstrates how the same puzzle can be tackled at different points on the spectrum between exhaustive search and fully precomputed optimal policy.

## 1 Introduction

Combinatorial puzzles such as the 8-puzzle, 15-puzzle, and Rubik’s Cube have long served as benchmark problems in the study of search algorithms and artificial intelligence. They are attractive because their rules are simple yet their state spaces grow exponentially with board size, making naive exhaustive search infeasible.

The **Twiddle Puzzle** is one such problem. On an  $N \times N$  board there are  $(N - 1)^2$  possible twiddle moves at every step, and the number of reachable configurations is at most  $(N^2)!$  (the number of permutations of  $N^2$  tiles). For a  $3 \times 3$  board this is already 362,880 states; for  $4 \times 4$  it exceeds 20 trillion.

This project focuses on two classical algorithmic paradigms that address this challenge in complementary ways:

- **Backtracking** — systematically explores the move tree, pruning branches that cannot lead to a solution within a given depth budget. It requires little memory but may re-explore the same state many times.
- **Dynamic Programming** — trades memory for speed by storing the result of every sub-problem exactly once. Given enough preprocessing time, each subsequent query is answered in constant time.

The project implements three solvers (Cases 6, 7, and 8 in `Main.java`) that span this spectrum, and evaluates their trade-offs empirically on  $3 \times 3$  and  $4 \times 4$  boards.

## 2 Problem Definition

### 2.1 Puzzle Description

The Twiddle Puzzle is played on an  $N \times N$  grid of numbered tiles. Tiles are labelled 1 through  $N^2$  and are initially arranged in a random permutation. The puzzle is **solved** when the tiles read  $1, 2, 3, \dots, N^2$  in left-to-right, top-to-bottom order.

The only legal operation is a **twiddle move**: select any  $2 \times 2$  sub-grid and rotate its four tiles  $90^\circ$  **counter-clockwise**. On an  $N \times N$  board there are  $(N-1)^2$  such sub-grids, numbered 1 to  $(N-1)^2$ .

## 2.2 State Representation

A puzzle state is represented as a two-dimensional integer array `int[][] grid` of size  $N \times N$ . Each cell holds the tile number currently occupying that position. States are serialised into a unique string key for use in hash maps and visited sets:

```
private String encode(int[][] g) {
    StringBuilder sb = new StringBuilder();
    for (int[] r : g)
        for (int x : r)
            sb.append(x).append(',');
    return sb.toString();
}
```

For example, the solved  $3 \times 3$  state encodes as "1,2,3,4,5,6,7,8,9,".

## 2.3 Initial State

The board is initialised by shuffling the numbers  $1 \dots N^2$  randomly using `Collections.shuffle()` inside `Board.generateRandomBoard()`. This guarantees a uniformly random starting permutation each time a new game begins.

## 2.4 Goal State

The goal is the unique configuration where tile  $k$  occupies position  $(\lfloor (k-1)/N \rfloor, (k-1) \bmod N)$ :

$$\text{goal}[i][j] = i \cdot N + j + 1, \quad 0 \leq i, j < N$$

## 2.5 Allowed Moves

Move  $m$  ( $1 \leq m \leq (N-1)^2$ ) targets the  $2 \times 2$  sub-grid whose top-left corner is at row  $r = \lfloor (m-1)/(N-1) \rfloor$ , column  $c = (m-1) \bmod (N-1)$ . The rotation is **counter-clockwise**: each tile moves to the next counter-clockwise position in the  $2 \times 2$  sub-grid:

$$\begin{pmatrix} \text{TL} & \text{TR} \\ \text{BL} & \text{BR} \end{pmatrix} \xrightarrow{\text{counter-clockwise}} \begin{pmatrix} \text{TR} & \text{BR} \\ \text{TL} & \text{BL} \end{pmatrix}$$

The cycle of assignments is  $\text{TL} \leftarrow \text{TR} \leftarrow \text{BR} \leftarrow \text{BL} \leftarrow \text{TL}$ : each cell receives the value from its clockwise neighbour, which moves the tiles counter-clockwise overall.

In code (`Board.rotate2x2()`):

```
int tmp      = grid[r][c];      // save TL
grid[r][c]   = grid[r][c+1];    // TL <- TR
grid[r][c+1] = grid[r+1][c+1];  // TR <- BR
grid[r+1][c+1] = grid[r+1][c];  // BR <- BL
grid[r+1][c] = tmp;             // BL <- TL (saved)
```

The **inverse** (clockwise) rotation used by the bottom-up DP builder to walk backwards from the goal reverses this cycle:  $\text{TL} \leftarrow \text{BL} \leftarrow \text{BR} \leftarrow \text{TR} \leftarrow \text{TL}$ .

### 3 Algorithmic Approaches

This project implements three solvers that together cover both Backtracking and Dynamic Programming:

| Case | Class                    | Paradigm                            |
|------|--------------------------|-------------------------------------|
| 6    | ComputerPlayer3 + DPFlow | Bottom-Up DP                        |
| 7    | BacktrackingPlayer       | Pure Backtracking                   |
| 8    | TopDownDPPlayer          | Top-Down DP (memoised backtracking) |

#### 3.1 Case 7 — Backtracking AI (BacktrackingPlayer)

##### 3.1.1 Concept

Backtracking is a depth-first search strategy that incrementally builds candidate solutions and *abandons* (backtracks from) a partial solution as soon as it determines that it cannot be extended to a valid complete solution.

In the context of the Twiddle Puzzle, a “candidate solution” is a sequence of twiddle moves. The solver tries moves one at a time, recurses deeper, and undoes the move if the recursive call returns no solution.

##### 3.1.2 Iterative Deepening (IDDFS)

Rather than setting a fixed depth limit, the solver uses **Iterative Deepening Depth-First Search**: it runs a depth-limited backtracking search for  $\text{maxDepth} = 1, 2, 3, \dots, 10$ . This gives the memory efficiency of DFS while guaranteeing that the first solution found is the shallowest one reachable within the depth budget.

```
// From BacktrackingPlayer.getMove()
for (int maxDepth = 1; maxDepth <= 10; maxDepth++) {
    Set<String> visited = new HashSet<>();
    List<Integer> path = new ArrayList<>();
    if (backtrack(currentGrid, 0, maxDepth, path, visited))
        return path.get(0);    // first move of the found path
}
return getBestGreedyMove(currentGrid); // fallback
```

##### 3.1.3 Sub-Problem Definition

The backtracking solver implicitly answers the following decision sub-problem at every node of the search tree:

*“Is there a sequence of at most  $(\text{maxDepth} - \text{depth})$  twiddle moves that transforms board state  $s$  into the goal?”*

Formally, define:

$$\text{Solvable}(s, k) = \begin{cases} \text{true} & \text{if } s \text{ is the goal state} \\ \text{false} & \text{if } k = 0 \text{ and } s \neq \text{goal} \\ \bigvee_{m=1}^{(N-1)^2} \text{Solvable}(\text{rotate}(s, m), k - 1) & \text{otherwise} \end{cases}$$

where  $k = \text{maxDepth} - \text{depth}$  is the remaining move budget. The top-level call is `Solvable( $s_0$ , maxDepth)` for increasing values of maxDepth. The first move  $m^*$  that makes the recursive call return **true** is what `getMove()` returns.

### 3.1.4 Mapping to a Classical Problem

This is structurally identical to finding the **shortest path on an unweighted implicit graph** (equivalent to the BFS/IDA\* formulation):

- **Nodes** — board states (all permutations of  $N^2$  tiles).
- **Edges** — two states are connected if they differ by exactly one twiddle move; every edge has weight 1.
- **Goal** — find a minimum-length path from the scrambled start node to the unique goal node.

The iterative-deepening structure maps directly to **Iterative Deepening A\* (IDA\*)**, the canonical algorithm for bounded shortest-path on implicit graphs. The heuristic  $h(s)$  plays the same role as in A\*: it lower-bounds the true distance to goal so that the condition  $\text{depth} + h(s) \leq \text{maxDepth}$  is never violated for any node on an optimal path, preserving optimality within each depth limit.

### 3.1.5 Pruning Strategies

Three pruning mechanisms reduce the search space. Together they can eliminate the vast majority of nodes that a naive DFS would visit.

**Pruning 1 — Admissible Heuristic Bound** *What it prunes:* Any partial path whose length plus the minimum possible remaining distance already exceeds the current depth limit.

The heuristic counts misplaced tiles and divides by 4, because one twiddle simultaneously touches exactly 4 cells and can fix at most 4 tiles in one move:

$$h(s) = \left\lceil \frac{|\{(i, j) : s[i][j] \neq \text{goal}[i][j]\}|}{4} \right\rceil$$

This is **admissible** — it never overestimates — so no optimal solution is ever discarded. The pruning condition is:

$$\underbrace{\text{depth}}_{\text{moves made so far}} + \underbrace{h(s)}_{\text{lower bound on remaining moves}} > \text{maxDepth} \implies \text{cut this entire subtree}$$

In code (`BacktrackingPlayer.backtrack()`):

```
if (depth + estimateRemaining(grid) > maxDepth)
    return false;    // cannot reach goal within budget
```

*Example:* If 8 tiles are misplaced,  $h = 2$ . At  $\text{depth} = 4$  and  $\text{maxDepth} = 5$ , this fires immediately and prunes  $(N - 1)^{2 \times 1}$  nodes in one check instead of expanding them.

**Pruning 2 — Cycle Detection (Path Constraint)** *What it prunes:* Paths that revisit a state already on the active call stack, preventing infinite loops.

A visited set tracks states on the *current root-to-node path*. The state is **removed when backtracking**, allowing the same state to appear on a completely different path:

```
if (visited.contains(stateKey)) return false; // cycle      prune

visited.add(stateKey);
// ... recurse through child moves ...
visited.remove(stateKey); // restore on backtrack
```

*Why not a global visited set?* A globally visited state might still be reachable optimally via a different, shorter path. A global set would make the algorithm incomplete. The per-path set cuts all cycles of length  $\leq \text{maxDepth}$  while preserving completeness.

**Pruning 3 — Move Ordering (Forward Heuristic Guidance)** *What it reduces:* Not branches outright, but the number of backtracks before the first solution is found, by trying the best-looking moves first.

All  $(N - 1)^2$  successors are generated, scored by  $h$ , and sorted ascending before any recursive call:

```
List<MoveOption> options = new ArrayList<>();
for (int m = 1; m <= board.totalMoves(); m++) {
    int[][] nextGrid = rotateCopy(grid, m);
    options.add(new MoveOption(m, estimateRemaining(nextGrid),
                               nextGrid));
}
options.sort(Comparator.comparingInt(o -> o.score)); // lowest h
first
```

*Impact:* If the optimal move reduces misplaced tiles from 8 to 4 and is sorted to position 1, the correct path is found on the first try with zero backtracking on that level. In the best case the effective branching factor drops from  $(N - 1)^2$  toward 1.

### 3.1.6 Backtracking Pseudocode

```
Backtrack(grid, depth, maxDepth, path, visited):
    if isSolved(grid):
        return true

    if depth + h(grid) > maxDepth:
        return false                // heuristic prune

    if encode(grid) in visited:
        return false                // cycle prune

    visited.add(encode(grid))

    successors = [(m, nextGrid) for m in moves]
    sort successors by h(nextGrid) ascending

    for each (move, nextGrid) in successors:
```



```

    path.add(move)
    if Backtrack(nextGrid, depth+1, maxDepth, path, visited):
        return true
    path.removeLast() // backtrack

visited.remove(encode(grid))
return false

```

### 3.1.7 Greedy Fallback

If no solution is found within depth 10, `getBestGreedyMove()` is called. It tries every move from the current state and returns the one that produces the fewest misplaced tiles, breaking ties by random selection.

### 3.1.8 Time and Space Complexity

Let  $M = (N - 1)^2$  be the number of twiddle moves per step and  $D$  be the depth of the shallowest solution found within the budget.

- **Time complexity:**  $O(M^D)$  in the worst case. At each of the  $D$  levels the solver may expand all  $M$  successors before finding the correct one, giving a search tree of size  $M^D$ . The heuristic pruning and move ordering reduce the *effective* branching factor substantially in practice, but the asymptotic worst case remains exponential. Across the outer IDDFS loop (depths  $1, 2, \dots, D$ ) the dominant cost is the last iteration, so the overall per-`getMove()` cost is still  $O(M^D)$ .
- **Space complexity:**  $O(D)$ . DFS stores only the current root-to-leaf path in the call stack and the `visited` set (at most  $D$  entries at any one time). No additional structures grow with the state space size  $|\mathcal{S}|$ . This is the primary advantage of backtracking over the DP approaches.
- **Concrete figures:**
  - $3 \times 3$  ( $M = 4$ , typical  $D \approx 10$ – $14$ ): worst case  $\sim 4^{14} \approx 268$  million nodes, but pruning reduces explored nodes to tens of thousands in practice.
  - $4 \times 4$  ( $M = 9$ , typical  $D \approx 20$ – $40$ ): the exponential blowup makes deeper solutions increasingly costly, which is why the greedy fallback is triggered frequently on larger boards.

## 3.2 Case 8 — Top-Down DP (TopDownDPPlayer)

### 3.2.1 Concept

Top-Down Dynamic Programming is backtracking augmented with **memoisation**. Every time a sub-problem (state, remaining depth) is solved, the result is stored in a table. If the same sub-problem is encountered again on a different recursive branch, it is answered from the table in  $O(1)$  rather than being re-explored.

This is the standard transformation that converts an exponential backtracking solution into a polynomial DP solution wherever overlapping sub-problems exist.

### 3.2.2 Sub-Problem Definition

Each unique call to `topDownDP` constitutes one DP sub-problem:

*“Given board state  $s$  and remaining depth budget  $k$ , what is the first move  $m^*$  that leads to the goal within  $k$  steps — or does no such move exist?”*

Formally, define  $f(s, k)$  as the first valid move from  $s$  with budget  $k$ :

$$f(s, k) = \begin{cases} 0 & \text{if } s = \text{goal} \\ -1 & \text{if } k = 0 \text{ and } s \neq \text{goal} \\ -1 & \text{if } h(s) > k \quad (\text{admissible prune}) \\ -1 & \text{if } s \in \text{currentPath} \quad (\text{cycle prune}) \\ \min\{m : f(\text{rotate}(s, m), k - 1) \neq -1\} & \text{otherwise} \end{cases}$$

The answer  $-1$  means “unsolvable within budget  $k$ ”; any positive integer is the first move of a valid path. The memo table stores  $f(s, k)$  so that each  $(s, k)$  pair is computed at most once across all recursive branches.

### 3.2.3 Mapping to a Classical Problem

Top-Down DP on the Twiddle Puzzle maps directly to the classical **Shortest Path with Memoisation** problem, which is itself a specialisation of the general **Optimal Substructure + Overlapping Sub-problems** DP template:

- **Optimal substructure:** The shortest path from  $s$  to goal of length  $k$  consists of a first move  $m$  followed by a shortest path from  $\text{rotate}(s, m)$  to goal of length  $k - 1$ . This is identical to the recurrence in **Bellman-Ford** / single-source shortest path on an unweighted DAG.
- **Overlapping sub-problems:** Many different move sequences lead to the same intermediate board state  $s'$  with the same remaining budget  $k'$ . Without memoisation each such occurrence triggers a full re-expansion (Case 7 behaviour). With memoisation (Case 8),  $f(s', k')$  is computed once and reused.
- **Classical analogue:** This is the **bounded shortest path** (or *k-step reachability*) problem, equivalent to computing the  $k$ -th power of a graph’s adjacency matrix entry  $A^k[\text{start}][\text{goal}]$ , solved efficiently by DP rather than repeated matrix multiplication.

### 3.2.4 The Memo Key

Two visits to the same board state are only equivalent if they have the same *remaining depth budget*. A state reached with 5 moves left may be solvable, while the same state with only 1 move left may not be. Both dimensions must be encoded:

```
String memoKey = encode(grid) + ":" + (maxDepth - currentDepth);
```

### 3.2.5 DP Recurrence

Using  $k = \text{maxDepth} - \text{currentDepth}$  as the remaining budget (consistent with the sub-problem definition  $f(s, k)$  above):

$$f(s, k) = \begin{cases} 0 & \text{if } s = \text{goal} \\ -1 & \text{if } k = 0 \text{ and } s \neq \text{goal} \\ -1 & \text{if } h(s) > k \quad (\text{admissible prune: bud} \\ -1 & \text{if } s \in \text{currentPath} \quad (\text{cycle detected}) \\ \min\{m \in \{1, \dots, M\} : f(\text{rotate}(s, m), k - 1) \neq -1\} & \text{otherwise} \end{cases}$$

A return value of  $-1$  means no solution exists within budget  $k$ ; any value  $m \geq 1$  is the first move of a valid path. The memo table stores  $f(s, k)$  keyed as `encode(s) + ":" + k` so each  $(s, k)$  pair is solved at most once.

### 3.2.6 Top-Down DP Pseudocode

```

TopDownDP(grid, depth, maxDepth, memo, visitedPath):
    if isSolved(grid):
        return 0                                // goal reached

    if depth + h(grid) > maxDepth:
        return -1                                // heuristic prune

    key = encode(grid) + ":" + (maxDepth - depth)
    if key in memo:
        return memo[key]                        // DP lookup: O(1)

    if encode(grid) in visitedPath:
        return -1                                // cycle prune

    visitedPath.add(encode(grid))
    result = -1

    for m = 1 to totalMoves:
        nextGrid = rotate(grid, m)
        if TopDownDP(nextGrid, depth+1, maxDepth, memo,
            visitedPath) != -1:
            result = m
            break

    visitedPath.remove(encode(grid))
    memo[key] = result                            // DP store
    return result

```

### 3.2.7 Difference from Case 7

| Case 7 — Backtracking                                                      | Case 8 — Top-Down DP                                             |
|----------------------------------------------------------------------------|------------------------------------------------------------------|
| Re-explores same (state, depth) every time it is encountered on a new path | Stores result of (state, depth) permanently; never recomputes it |
| <code>visited.remove()</code> clears state after backtrack                 | <code>memo.put()</code> permanently records the outcome          |
| Exponential worst-case per call                                            | Polynomial when many transpositions exist                        |

Table 1: Backtracking (Case 7) vs Top-Down DP (Case 8)

### 3.2.8 Time and Space Complexity

Let  $|\mathcal{S}|$  be the number of distinct reachable board states,  $M = (N - 1)^2$  the branching factor, and  $D$  the solution depth.

- **Time complexity:**  $O(|\mathcal{S}| \cdot D)$  in the worst case. The memo table has at most  $|\mathcal{S}| \times D$  distinct keys (one per (state, remaining-budget) pair), and each entry is computed at most once at cost  $O(M)$  per entry. Hence the total work is  $O(|\mathcal{S}| \cdot D \cdot M)$ ; since  $M$  is a small constant for fixed  $N$  this is written  $O(|\mathcal{S}| \cdot D)$ . This is polynomial in  $|\mathcal{S}|$ , a significant improvement over the  $O(M^D)$  exponential cost of Case 7 when many transpositions exist.
- **Space complexity:**  $O(|\mathcal{S}| \cdot D)$ . The memo table may accumulate one entry per (state, budget) pair encountered during the search. In addition, the call stack holds  $O(D)$  frames and the per-path `visited` set holds at most  $O(D)$  entries, both of which are dominated by the memo table.
- **Concrete figures:**
  - $3 \times 3$  ( $|\mathcal{S}| \leq 362,880$ ,  $D \leq 10$ ): memo table bounded at  $\approx 3.6$  million entries; in practice far fewer are reached due to pruning.
  - $4 \times 4$  ( $|\mathcal{S}|$  in the trillions): the theoretical bound is enormous, but the depth cap of 10 and heuristic pruning keep the actual memo size manageable for moderately scrambled boards.

## 3.3 Case 6 — MDF DP (ComputerPlayer3 + DPFlow)

### 3.3.1 Concept

The **Minimum-Depth-First Dynamic Programming** (MDF DP) solver takes the opposite direction from Cases 7 and 8. Rather than searching forward from a scrambled state at query time, it performs an expensive one-time **offline preprocessing** step that builds a complete lookup table, after which every query is answered in  $O(1)$ .

### 3.3.2 Sub-Problem Definition

Each entry in the DP table is the answer to:

*“What is the minimum number of twiddle moves needed to reach the goal from board state  $s$ ?”*

Formally:

$\text{dist}(s)$  = minimum number of moves to transform  $s$  into the goal state

with base case  $\text{dist}(\text{goal}) = 0$ . Unlike Cases 7 and 8, there is *no depth parameter* — the sub-problem depends only on the state  $s$ , not on any budget constraint. This is possible because the bottom-up BFS guarantees that every state is first reached via the shortest path, so no re-examination is needed.

A second table records the optimal first action:

$$\text{bestMove}(s) = \arg \min_m \text{dist}(\text{rotate}(s, m))$$

### 3.3.3 Mapping to a Classical Problem

This is a direct implementation of **Single-Source Shortest Path on an Unweighted Graph** (i.e., plain BFS), which is itself the simplest instance of **Bellman-Ford DP**:

- **Graph:** Nodes = board states; edges = twiddle moves (weight 1 each).
- **Source:** The goal state (BFS is run *backwards* from goal so that  $\text{dist}(s)$  gives the forward distance from any  $s$  to goal).
- **Classical analogue:** The **BFS shortest-path** problem. The DP recurrence is identical to the standard BFS relaxation:  $d[v] = d[u] + 1$  if edge  $(u, v)$  exists and  $d[v]$  is not yet set. This is also the same structure as the **coin-change** or **word-ladder** DP: minimum number of unit-cost steps to reach a target state.
- **Optimal substructure:**  $\text{dist}(s) = 1 + \text{dist}(s')$  where  $s'$  is the neighbour of  $s$  closest to goal. The optimal solution to the whole problem contains optimal solutions to each sub-state along the path.
- **No overlapping sub-problems in the usual sense:** BFS processes each state exactly once, so there is no redundant recomputation. The DP “table” is the BFS visited set combined with the distance array.

### 3.3.4 Bottom-Up Table Construction

The table is built by BFS *backwards* from the goal, applying **inverse** twiddle moves. Because the forward twiddle is a counter-clockwise rotation, its inverse is a **clockwise** rotation:

Forward (counter-clockwise):  $\text{TL} \leftarrow \text{TR} \leftarrow \text{BR} \leftarrow \text{BL} \leftarrow \text{TL}$

Inverse (clockwise):  $\text{TL} \leftarrow \text{BL} \leftarrow \text{BR} \leftarrow \text{TR} \leftarrow \text{TL}$

Starting from the goal state with distance 0, every state reachable by one inverse (clockwise) move is at distance 1, and so on. Two tables are populated:

$$\begin{aligned}\text{distance}[\text{state}] &= \text{minimum moves from state to goal} \\ \text{bestMove}[\text{state}] &= \text{move to apply to reach goal optimally}\end{aligned}$$

### 3.3.5 DP Recurrence Equation

Let  $M = (N - 1)^2$  be the number of twiddle moves and  $\mathcal{S}$  be the set of all reachable board states. The full recurrence is:

$$\text{dist}(s) = \begin{cases} 0 & \text{if } s = \text{goal} \\ 1 + \min_{m=1}^M \text{dist}(\text{rotate}(s, m)) & \text{if } s \in \mathcal{S} \setminus \{\text{goal}\} \end{cases}$$

Because the graph is unweighted, all successors have equal cost, so the minimum reduces to choosing the neighbour with the smallest already-known distance. The bottom-up BFS computes this by processing states in non-decreasing order of distance (BFS layer = distance level), so when state  $s$  is first dequeued,  $\text{dist}(s)$  is already optimal.

The companion policy table is:

$$\text{bestMove}(s) = \arg \min_{m \in \{1, \dots, M\}} \text{dist}(\text{rotate}(s, m))$$

*Complexity:*

- Time:  $O(|\mathcal{S}| \cdot M)$  — each state is dequeued once and  $M$  inverse moves are applied.
- Space:  $O(|\mathcal{S}|)$  — two hash maps of size  $|\mathcal{S}|$ .
- For  $3 \times 3$ :  $|\mathcal{S}| \leq 362,880$ ,  $M = 4 \Rightarrow$  table built in  $\approx 150$  ms.
- For  $4 \times 4$ :  $|\mathcal{S}|$  is capped at 2,000,000 due to memory constraints.

### 3.3.6 Bottom-Up DP Pseudocode

```
BuildDPTable(board):
    queue = {goal}
    distance[goal] = 0

    while queue not empty:
        state = dequeue(queue)
        d = distance[state]

        for m = 1 to totalMoves:
            prev = inverseRotate(state, m)
            if prev not in distance:
                distance[prev] = d + 1
                bestMove[prev] = m
                enqueue(queue, prev)
            if |distance| >= MAX_STATES:
```

```

        return (truncated table)

return (distance, bestMove)

```

The table is capped at 2,000,000 states to bound memory usage. For  $3 \times 3$  boards the complete reachable graph (at most 362,880 states) fits entirely. For  $4 \times 4$  boards the table is necessarily partial.

### 3.3.7 Query Phase via DPFlow

Once the table is built, `DPFlow.Session.playNextMove()` answers each move request in  $O(1)$ :

```

// From ComputerPlayer3.chooseMove()
Integer direct = data.bestMove.get(encode(grid));
if (direct != null) return direct;          // O(1) table lookup

// Fallback for partial table: pick neighbour with lowest
// distance
for (int m = 1; m <= board.totalMoves(); m++) {
    int[][] next = rotateCopy(grid, m);
    Integer d = data.distance.get(encode(next));
    if (d != null && d < bestDist) { bestDist = d; bestMove = m;
    }
}

```

### 3.3.8 Reshuffle Mechanism

Because the table may be partial (truncated), some random starting boards may not appear in it. `DPFlow.initialize()` handles this by re-randomising the board up to 2,000 times until a solvable configuration is found:

```

while (!session.isSolvable(board) && reshuffles < maxReshuffles)
{
    board.randomize();
    reshuffles++;
}

```

### 3.3.9 Time and Space Complexity

The MDF DP solver has two clearly separated cost phases that must be analysed independently.

#### Preprocessing — Table Build

- **Time complexity:**  $O(|\mathcal{S}| \cdot M)$  where  $|\mathcal{S}|$  is the number of reachable states and  $M = (N - 1)^2$  is the number of inverse twiddle moves applied per state. Each state is enqueued and dequeued exactly once (standard BFS guarantee), and  $M$  inverse-rotate operations are performed per dequeue.

- **Space complexity:**  $O(|\mathcal{S}|)$  for the two hash maps (`distance` and `bestMove`), each storing one entry per visited state. The BFS queue holds at most  $O(|\mathcal{S}|)$  entries at peak (the wavefront of the current BFS layer).
- **Concrete figures:**
  - $3 \times 3$ :  $|\mathcal{S}| \leq 362,880$ ,  $M = 4 \Rightarrow \approx 1.5$  million operations; builds in  $\approx 150$  ms with  $\sim 30$  MB heap.
  - $4 \times 4$ : capped at  $|\mathcal{S}| = 2,000,000$ ,  $M = 9 \Rightarrow \approx 18$  million operations; builds in 30–90 s using up to 1.5 GB heap.

### Query Phase — Per Move

- **Time complexity:**  $O(1)$  per `getMove()` call when the current board state is in the table (direct `HashMap.get` lookup). For states outside the truncated table, the fallback scans all  $M$  neighbours, giving  $O(M)$  — still constant for fixed  $N$ .
- **Space complexity:**  $O(1)$  additional space at query time; the table itself was already allocated during preprocessing.
- This  $O(1)$  query cost is the defining advantage of the MDF DP approach: once built, the table can answer an unlimited number of move requests without any further search.



## 4 Comparison of DP vs Backtracking

| Feature                                              | Backtracking<br>(Case 7)                                         | Top-Down<br>(Case 8)                                                   | DP                 | Bottom-Up<br>(Case 6)                                   | DP            |
|------------------------------------------------------|------------------------------------------------------------------|------------------------------------------------------------------------|--------------------|---------------------------------------------------------|---------------|
| Core paradigm                                        | Recursive with undo                                              | DFS                                                                    | Memoised recursion | recur-                                                  | BFS from goal |
| Setup cost                                           | None                                                             | None                                                                   |                    | High (BFS pre-processing)                               |               |
| Per-move query                                       | Re-searches from scratch                                         | Partial reuse via memo                                                 |                    | $O(1)$ table lookup                                     |               |
| <b>Time complexity</b> (per <code>getMove()</code> ) | $O(M^D)$ where $M = (N-1)^2$ moves and $D$ is the solution depth | $O( \mathcal{S}  \cdot D)$ distinct $(s, k)$ pairs, each computed once |                    | $O(1)$ query; pre-processing $O( \mathcal{S}  \cdot M)$ |               |
| <b>Space complexity</b>                              | $O(D)$ — only the current path is stored                         | $O( \mathcal{S}  \cdot D)$ — memo table entries                        |                    | $O( \mathcal{S} )$ — two hash maps                      |               |
| Optimality                                           | Not guaranteed                                                   | Not guaranteed                                                         |                    | Guaranteed (if in table)                                |               |
| Memory usage                                         | Low (path only)                                                  | Medium (grows with unique sub-problems)                                |                    | High (up to 2M entries)                                 |               |
| Scales to $4 \times 4$                               | Slowly                                                           | Better than Case 7                                                     |                    | Partially (truncated table)                             |               |
| Handles unseen states                                | Always                                                           | Always                                                                 |                    | Requires reshuffle                                      |               |

Table 2: Detailed Comparison including Time and Space Complexity

**Notation:**  $M = (N - 1)^2$  is the branching factor (number of twiddle moves per step),  $D$  is the depth of the solution found, and  $|\mathcal{S}|$  is the number of reachable board states ( $\leq 362,880$  for  $3 \times 3$ , capped at 2,000,000 for  $4 \times 4$ ).

**When to prefer Backtracking (Case 7):** When memory is severely constrained, when the board size is small enough that  $\text{depth} \leq 10$  covers all cases, or when no pre-processing time is available. Its  $O(D)$  space cost makes it the only viable option in memory-limited environments.

**When to prefer Top-Down DP (Case 8):** When the same sub-states are likely to be revisited across different paths (high transposition rate), and moderate memory is available. The  $O(|\mathcal{S}| \cdot D)$  memo table trades memory for a dramatic reduction in the effective branching factor.

**When to prefer MDF DP (Case 6):** When the board size is small (ideally  $3 \times 3$ ), preprocessing time is acceptable, and every subsequent query must be answered in real time. Its  $O(1)$  per-move cost is the only approach that scales to repeated queries on the same board without re-computation.

## 5 Challenges Faced

### 5.1 Large State Spaces

The  $4 \times 4$  board has over 20 trillion possible states, making complete exploration impossible. The MDF DP table had to be capped at 2,000,000 states. This was addressed by implementing a partial-table fallback in `ComputerPlayer3.chooseMove()` that greedily picks the neighbour with the lowest known distance.

### 5.2 Avoiding Repeated States

Both backtracking solvers use a per-path `visited` set to detect cycles. The key design decision was to *remove* states from this set on backtracking (unlike a global visited set), which allows the same state to be revisited on a different path — necessary for completeness but potentially costly for performance.

### 5.3 Correct Memo Key Design (Case 8)

Early versions of Top-Down DP used only the board encoding as the memo key. This was incorrect because the same state reached with different remaining depth budgets has different solvability. The fix was to include the remaining depth in the key: `encode(grid) + ":" + (maxDepth - depth)`.

### 5.4 Inverse Rotation for Bottom-Up DP (Case 6)

Building the DP table requires walking *backwards* from the goal. Since the forward twiddle move is a counter-clockwise rotation, the inverse is a **clockwise** rotation. Getting the inverse cycle order correct ( $TL \leftarrow BL \leftarrow BR \leftarrow TR \leftarrow TL$ ) was a subtle source of bugs during development.

### 5.5 Thread Safety and Cancellation

The GUI runs each solver in a `SwingWorker` background thread. All three solvers check `Thread.currentThread().isInterrupted()` at the start of every loop iteration so that the user can cancel a long-running search without the application freezing.

### 5.6 Reshuffle Logic for Partial DP Table

When the MDF DP table is truncated, some random boards fall outside it. The reshuffle mechanism in `DPFlow.initialize()` re-randomises the board and retries, but this can degrade user experience if many reshuffles are needed. This was mitigated by logging the reshuffle count and warning the user if the solver is operating on a partial policy.

## 6 Conclusion

This project implemented and compared three puzzle-solving strategies for the Twiddle Puzzle, each representing a distinct point on the algorithmic spectrum:

- **Case 6 (MDF DP)** demonstrated that bottom-up dynamic programming, by inverting the search direction and precomputing the full optimal policy, achieves constant-time queries and provably optimal solutions — at the cost of significant upfront memory and time investment.
- **Case 7 (Backtracking AI)** showed that iterative-deepening backtracking with a good admissible heuristic and move ordering can solve moderately complex instances with minimal memory, but struggles as board size grows.
- **Case 8 (Top-Down DP)** illustrated the classic transformation from backtracking to dynamic programming: adding a memo table keyed on (state, remaining-depth) eliminates redundant sub-problem computation and consistently outperforms pure backtracking.

The key insight from the project is that all three approaches solve the same recurrence — the minimum-move path from a state to the goal — but differ in *when* and *how often* they compute each sub-problem’s answer. Bottom-up DP computes everything once offline; top-down DP computes on demand but remembers; pure backtracking recomputes every time.

Future work could explore combining the A\* heuristic with the DP table for even faster  $4 \times 4$  solving, or applying pattern databases to compute tighter lower bounds for the backtracking pruning step.

## References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- R. E. Korf, “Depth-first iterative-deepening: An optimal admissible tree search,” *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.
- S. Russel and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.
- Lecture notes for Design and Analysis of Algorithms.